UniSENAI

Estruturas de Dados

Aula 03: Introdução à análise de complexidade de algoritmos, com foco em notações assintóticas (O, Ω, Θ).

Matheus Luan Krueger, M.e

UniSENAI

Última aula...

- Introdução à recursividade: estrutura, funcionamento e exemplos clássicos (fatorial, sequência de Fibonacci, torres de Hanói).
- Comparação entre abordagens iterativas e recursivas.

UniSENAI

Roteiro

- Análise da complexidade de algoritmos
- Crescimento e comportamento de funções
- Classes de comportamento
- Elementos da análise assintótica

UniSENAI

Complexidade de algoritmos

- Consiste na quantidade de "trabalho" necessária para a sua execução, expressa

Vamos considerar o **tamanho da entrada**, comumente referenciada como **'n'**

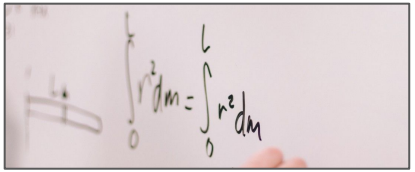
Para então avaliar o **"esforço" necessário** na etapa de processamento

ENTRADA | PROCESSAMENTO | SAÍDA

UniSENAI

Como comparar duas ou mais soluções?

- **Análise teórica**
  - Analisar o código com o devido cuidado e profundidade, avaliar números, expressões e compreender resultados.
- **Análise matemática**
  - Avaliar o algoritmo a partir de provas e funções matemáticas.



- **Benchmark (*análise empírica*)**
  - Executar dois ou mais algoritmos e verificar qual é o mais eficiente ou mais rápido.
  - Necessita de implementação de todas as opções.
  - Resultados podem variar de acordo com o ambiente de teste.
  - Alguns problemas são intratáveis computacionalmente.
- **Análise de algoritmos**
  - Avaliação de algum parâmetro ou característica de um algoritmo.
  - Simplicidade / Clareza
  - Eficiência
    - Espaço de memória (primária ou secundária)
    - Tráfego de rede
    - Tempo

UniSENAI

Como comparar duas ou mais soluções?

- Tomemos duas soluções para localizar um elemento em um vetor: LIN e BIN
- Temos dois computadores diferentes, A e B, sendo A um **Core i3** e B um **Core i7**.

Tamanho (n)	Tempo de execução de LIN em A	Tempo de execução de BIN em B
16	8 ns	100.000 ns
64	32 ns	150.000 ns
256	128 ns	200.000 ns
1024	512 ns	250.000 ns

Qual é a melhor solução?

LIN ou BIN

UniSENAI

Como comparar duas ou mais soluções?

Tamanho (n)	Tempo de execução de LIN em A	Tempo de execução de BIN em B
16	8 ns	100.000 ns
64	32 ns	150.000 ns
256	128 ns	200.000 ns
1024	512 ns	250.000 ns
...	...	...
1.048.576	524.288 ns	500.000 ns
4.194.304	2.097.152 ns	550.000 ns
16.777.216	8.388.608 ns	600.000 ns
...	...	...
$63{,}072 \times 10^{12}$	$31{,}536 \times 10^{12}$ ns ou **1 ano**	1.375.000 ns ou cerca de **1,4 segundos**

Qual é a melhor solução?

LIN ou BIN?

UniSENAI

Complexidade de algoritmos

- Pode-se falar de dois tipos de complexidade de algoritmos :
  - **Complexidade Espacial**
    - Quantidade de recursos utilizados para resolver o problema.
  - **Complexidade Temporal**
    - Quantidade de tempo utilizado. Pode ser vista também como o número de instruções necessárias para resolver um determinado problema.
- Em ambos os casos, seja uma análise espacial ou temporal, **a complexidade é medida de acordo com o tamanho dos dados de entrada (n)**.
- Estamos interessados em calcular a **Complexidade Temporal** de um algoritmo!

UniSENAI

Complexidade de algoritmos

- Existem três perspectivas para análise de complexidade:
  - **Melhor Caso**
  - **Caso Médio**
  - **Pior Caso**
- Nas três perspectivas, trataremos como $f(n)$ a função que mede o tempo (ou quantidade de instruções) necessário para executar um algoritmo com problema de **tamanho n**.

UniSENAI

Análise do MELHOR CASO

- Definido pela letra grega **Ω (Ômega)**
- Exprime o **menor tempo de execução** ou o **melhor resultado** que podemos ter a partir da execução de um algoritmo para uma entrada de **tamanho n**.
- É pouco usado, por ter aplicação em poucos casos. Exemplo:
  - O algoritmo de pesquisa sequencial em um vetor tem complexidade $f(n) = \Omega(1)$
  - A análise assume que o número procurado seria **o primeiro** selecionado na lista.
  - **Abordagem otimista!**

UniSENAI

Análise do CASO MÉDIO

- Definido pela letra grega **θ (Theta)**
- Exprime o **tempo médio de execução**, obtido a partir da **média dos tempos de execução de todas as entradas** de tamanho n, ou baseado em probabilidade de determinada condição ocorrer.
- Muito difícil de determinar na maioria dos casos. Exemplo:
  - O algoritmo de pesquisa sequencial em um vetor tem complexidade $f(n) = \theta(n/2)$
  - Em média será necessário visitar **n/2** elementos do vetor até encontrar o elemento procurado.
  - **Melhor aproximação!**

UniSENAI

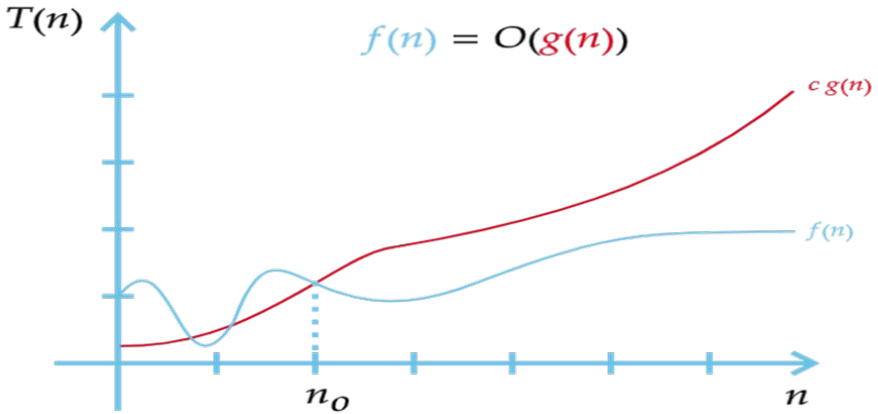
Análise do PIOR CASO

- Representado pela letra grega **O (ômicron maiúscula, big-O)**
- Expressa um **limite superior assintótico** para o crescimento do tempo de execução ou consumo de recursos de um algoritmo.
- Foca no **comportamento para valores grandes de n** , desconsiderando constantes e termos de menor ordem.
- O método mais utilizado para análise de complexidade, pois fornece uma estimativa confiável do desempenho de um algoritmo. Exemplo:
    - O algoritmo de busca sequencial em um vetor tem complexidade $O(n)$
    - No pior caso será necessário **visitar todos os n elementos** do vetor até encontrar (ou não) o elemento procurado.
    - **Abordagem pessimista!**

UniSENAI

A notação “big-O” limite assintótico superior

- Dadas funções assintoticamente não negativas f e g, dizemos que f está na ordem O de g e escrevemos $f = O(g)$ se: $f(n) <= c*g(n)$ para algum c positivo e para todo n suficientemente grande.
- Assim, diz-se que uma função limite superior $g(n)$ domina assintoticamente outra função $f(n)$ se existem duas constantes positivas c e n_0 tais que, para qualquer $n > n_0$ temos, $f(n) <= c*g(n)$. Se escreve que $f(n) = O(g(n))$ ou $f(n) \in O(g(n))$ para expressar essa dominação.



UniSENAI

A notação “Big O”

- *“A notação Big O é uma notação matemática que descreve o comportamento limitante de uma função quando o argumento tende a um valor específico ou ao infinito.”*
- **Tempo** (ou espaço) é contabilizado em **número de passos do algoritmo**.
- Análise do algoritmo determina uma função que depende do tamanho da entrada n.
- Exemplo: $10n^3 + 4n + 10$
  - **À medida que n aumenta, o termo cúbico começa a dominar**
  - A constante do termo cúbico tem relativamente a mesma importância que a velocidade da CPU, isto é, ambos alteram o tempo absoluto de execução, mas não mudam a complexidade assintótica.

UniSENAI

A

- te
- a
- as

n	$10*n^3$	4*n	+10	$10*n^3+4*n+10$
1	10	4	+10	24
2	80	8	+10	98
3	27	12	+10	292
4	640	16	+10	666
5	1250	20	+10	1.280
6	2160	24	+10	2.194
7	3430	28	+10	3.468
8	5120	32	+10	5.162
9	7290	36	+10	7.336
10	10000	40	+10	10.050
100	10000000	400	+10	10.000.410
1000	10000000000	4000	+10	10.000.004.010
10000	10000000000000	40000	+10	1.000.000.040.010
100000	10000000000000000	400000	+10	100.000.000.004.000.10
1000000	1000000000000000000	4000000	+10	10.000.000.000.040.000.10

À medida que n aumenta, o termo cúbico começa a dominar

UniSENAI

A notação “Big O”

- Desprezar constantes aditivas ou multiplicativas
  - Número de passos $3n$ será aproximado para n
- **Interesse no comportamento assintótico da função**
  - Desprezamos constantes multiplicativas, pois não alteram a taxa de crescimento
  - Desprezamos termos de menor grau, pois o termo de maior grau domina o crescimento da função.
    - $O(n^2 + n)$ será aproximado para $O(n^2)$
    - $O(6n^3 + 4n + 9)$ será aproximado para $O(n^3)$

Quando temos um **número de entrada gigantesco** ou o **valor de n tende ao infinito**, trabalhamos com a **análise assintótica.**

UniSENAI

Cálculo da complexidade

- Foi visto que, para calcular a complexidade de um algoritmo, iremos **analisar o pior caso**.
- A análise deve ser feita de acordo com a tabela a seguir:

Número de Operações	Complexidade
f(n)	$O(f(n))$
c x f(n)	$O(f(n))$
f(n) + f(n)	$O(f(n))$
f(n) + g(n)	$O(max(f(n),g(n))$
f(n) x g(n)	O(f(n) x g(n))

UniSENAI

Cálculo da complexidade de um algoritmo iterativo

algoritmo que resolve o problema de determinar o caminho mais curto entre todos os pares de nós em um grafo orientado e ponderado

```
void FloidWarshall(int dist[][]) {                                  1 +
        int i;                                                      1 +
        int j;                                                      1 +
        int k;
        for ( k = 0; k < n; k++ ) {                                 n * (
                for ( i = 0; i < n; i++ ) {                         n * (
                        for ( j = 0; j < n; j++ ) {                 n * (
                                int a = dist[i][j];                   1 +
                                int b = dist[j][k];                   1 +
                                int c = dist[k][j];                   1 +
                                dist[i][j] = min(a, b + c );          3
                        )
                )
        )
```

$$1 + 1 + 1 + n * (n * (n * (1 + 1 + 1 + 3)))$$

$$3 + n * n * n * 6$$

$$3 + 6n^3$$

$$\mathbf{O(n^3)}$$

```
                                                                    1 +
                                                                    1 +
                                                                    1 +
```

UniSENAI

Cálculo da função de complexidade

```
public static void main(String args[]) {
        ...     // código-fonte da entrada e definição de 'n' abstraído
        for (int i=0; i < n; i++) {
                int j = 4;
                j = 5 + j;
                j *= 2;
                System.out.println(j).
        }
        System.out.printl("acabou")
}
```

- Quando dizemos que a função linear $4n+1$ é $O(n)$, estamos afirmando que: "*o comportamento assintótico de uma função linear é igual ou superior ao dela*". Significa dizer:
    - **A função $4n+1$ nunca crescerá a ponto de ultrapassar o comportamento linear.**

UniSENAI

Principais classes de problemas

- Complexidade Constante
- Complexidade Linear
- Complexidade Logarítmica
- Complexidade Log Linear
- Complexidade Quadrática
- Complexidade Cúbica / Polinomial
- Complexidade Exponencial
- Complexidade Fatorial

1	*Constante*	O nirvana dos algoritmos. Na verdade, esta é uma situação muito improvável, pois ocorre em algoritmos que não realizam nenhuma computação significativa e, portanto, não possuem muita utilidade.

Complexidade Constante

- São os algoritmos onde **a complexidade independe do tamanho n de entradas**.
- É o único em que as instruções dos algoritmos são executadas um número fixo de vezes.

```
if (condição == true) {
        realiza alguma operação em tempo constante
} else {
        realiza alguma operação em tempo constante
}
```

Complexidade Linear O(n)

n *Linear*	Algoritmo muito eficiente. A classe linear ocorre em algoritmos que precisam processar n elementos de entrada e fazer pequenas computações sobre esses elementos.

- Uma **operação é realizada em cada elemento de entrada**, exemplo: pesquisa de elementos em uma lista.

```
for (i = 0; i < N; i = i + 1 ) {
        if (condição == true) {
                realiza alguma operação em tempo constante
        } else  {
                realiza alguma operação em tempo constante
        }
}
```

log n *Logarítmica*	O melhor dos mundos para os algoritmos. Essa classe é típica de algoritmos que podem resolver os problemas transformando-os em problemas menores a cada etapa da resolução.

Complexidade Logarítmica

- Ocorre tipicamente em **algoritmos que dividem o problema em problemas menores**.

```
int PesquisaBinaria ( int array[], int chave , int N){
        int inf = 0;
        int sup = N - 1;
        int meio;
        while (inf <= sup) {
                meio = (inf+sup)/2;
                if (chave == array[meio]) {
                        return meio;
                } else {
                        if (chave < array[meio]) { sup = meio-1; }
                        else { inf = meio+1; }
                }
        }
        return -1;   // não encontrado
}
```

n log n *Linear logarítmica*	Um pouco menos eficientes que os lineares. Essa é uma classe assintótica típica dos algoritmos que implementam o paradigma de "divisão e conquista".

Complexidade Log Linear

- Ocorre tipicamente em **algoritmos que dividem o problema em problemas menores, porém juntando posteriormente a solução dos problemas menores**.

```
void merge(int inicio, int fim) {
        if (inicio < fim) {
                int meio = (inicio + fim) / 2;
                merge(inicio, meio);
                merge(meio + 1, fim);
                mesclar(inicio, meio, fim);
        }
}
```

n^2 *Quadrática*	Polinomiais de ordem quadrática. Essa classe ocorre bastante em algoritmos que processam informações aos pares. Geralmente em *loops* aninhados (um *loop* dentro do outro).

Complexidade Quadrática

- Ocorre tipicamente em **algoritmos que dividem o problema em problemas menores, porém juntando posteriormente a solução dos problemas menores**.

```
void merge(int inicio, int fim) {
        if (inicio < fim) {
                int meio = (inicio + fim) / 2;
                merge(inicio, meio);
                merge(meio + 1, fim);
                mesclar(inicio, meio, fim);
        }
}
```

Complexidade Polinomial

n^k

Polinomial

São mais eficientes que os exponenciais, porém só são adequados para resolver pequenos problemas. Quando o valor de n cresce muito, o algoritmo se torna bastante ineficiente.

- Itens são processados três a três, **geralmente com um loop dentro do outros dois (ou mais)**.

```
int dist[N][N];
int i, j, k;

for ( k = 0; k < N; k++ )
        for ( i = 0; i < N; i++ )
                for ( j = 0; j < N; j++ )
                        dist[i][j] = min( dist[i][j], dist[i][k] + dist[k][j] );
```

Complexidade Exponencia

2^n	*Exponencial*	Mais eficiente que o fatorial, porém ainda muito longe de ser um algoritmo eficiente. Também ocorre em algoritmos que usam força bruta para resolver problemas.

- Utilização de **"Força Bruta"** para encontrar a solução de um problema.
- A solução geralmente é baseada diretamente no enunciado do problema e nas definições dos conceitos envolvidos.
- Exemplo:
  - Utilizando apenas números é possível criar 10^n senhas de n dígitos, pois temos 10 algarismos numéricos.
  - Um algoritmo de força bruta para quebrar uma dessas senhas tem complexidade $O(2^n)$

Complexidade Fatorial O(

n! *Fatorial*	A pior classe existente. Geralmente ocorre em alguns algoritmos que usam a técnica de força bruta para resolver problemas.

- Também é baseada na utilização de **força bruta** para encontrar a solução de um problema. **A classe fatorial consiste em testar todas as possíveis permutações existentes na solução, a procura da solução ótima para o problema**.
- Exemplo: **Problema do Caixeiro Viajante**
  - Encontrar a rota mínima para visitar várias cidades sem repetir nenhuma
  - É um problema base para o projeto de microchips, sequenciamento de genoma e muitas outras aplicações
  - Não possui solução exata eficiente (Problema NP)
  - Utilização de heurísticas para aproximar a solução ótima

UniSENAI

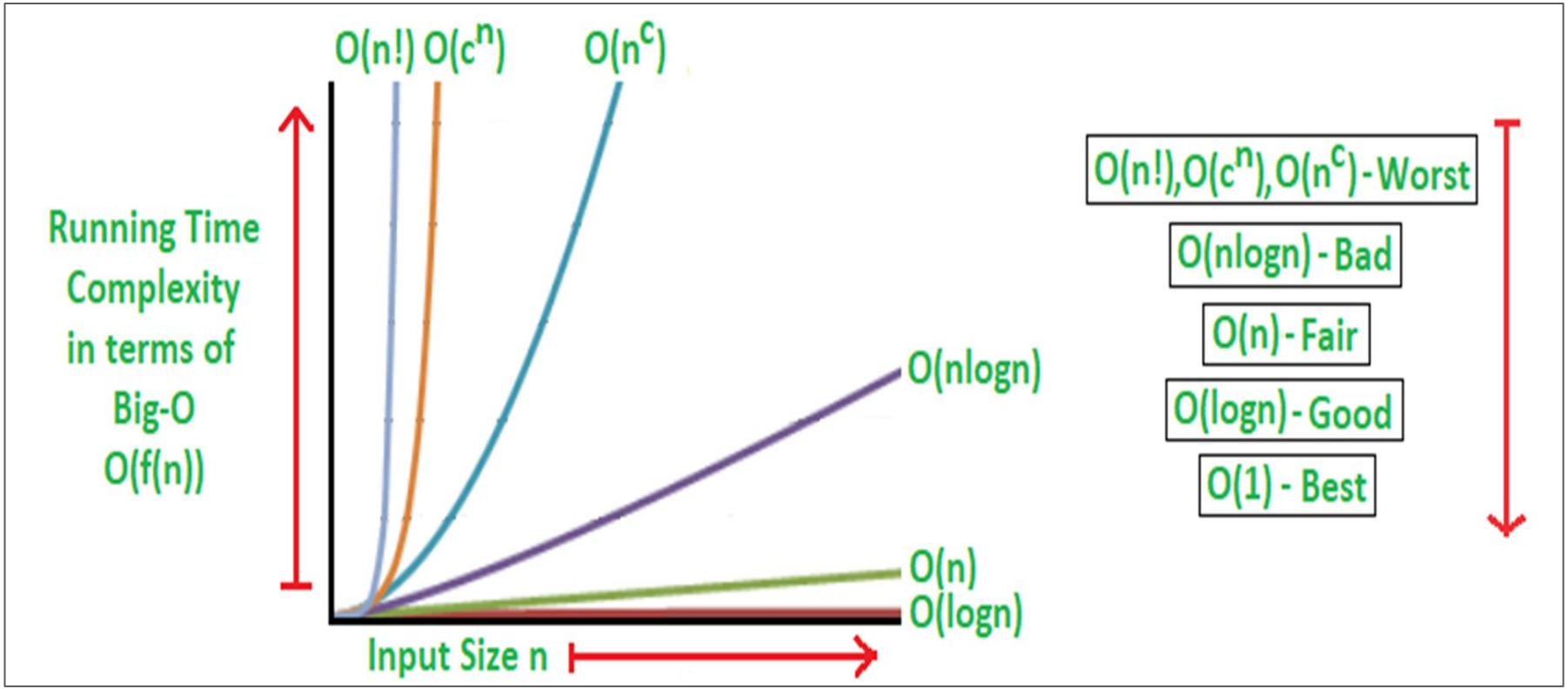
Ordens de complexidade

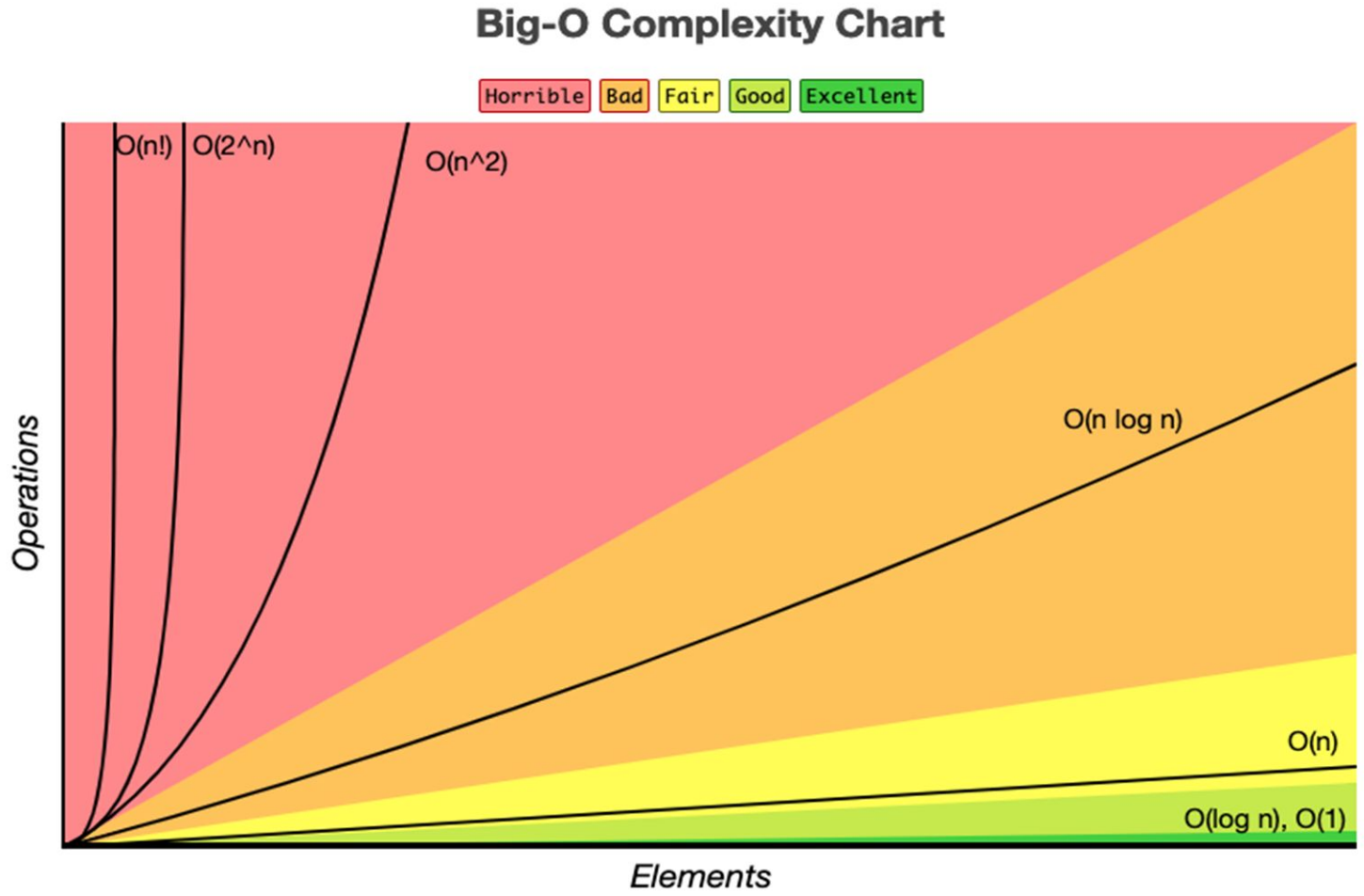
- Imagine um computador que leva 1ms para executar uma operação. A tabela abaixo indica o tempo aproximado de execução de um algoritmo com diferentes ordens de complexidades para 3 tamanhos de entrada:

n	O(n)	Log(n)	nLog(n)	$O(n^2)$	$O(n^k)$	$O(2^n)$	O(n!)
16	0.016s	0.004s	0.064s	0.256s	4s	1m5s	663 anos
32	0.032s	0.005s	0.160s	1s	33s	49 dias	10^{23} sec
512	0.512s	0.009s	4.608s	4m22s	37h	10^{143} sec	...

UniSENAI

Ordens de complexidade




Big-O Complexity Chart
Horrible
Bad
Fair
Good
Excellent
O(n!)
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n), O(1)
Operations
Elements

UniSENAI

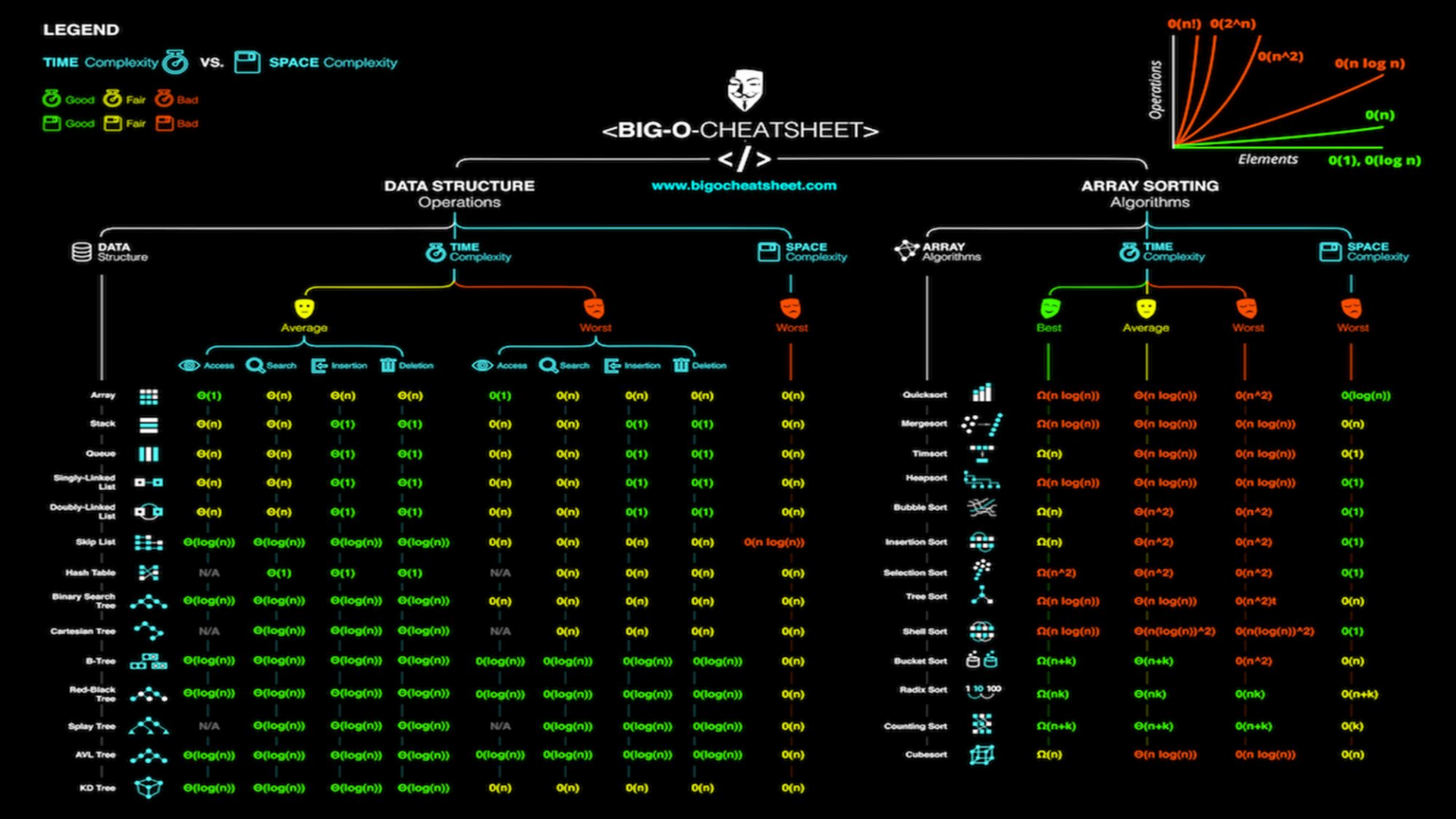
Common Data Structure Operations

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

UniSENAI

Array Sorting Algorithms

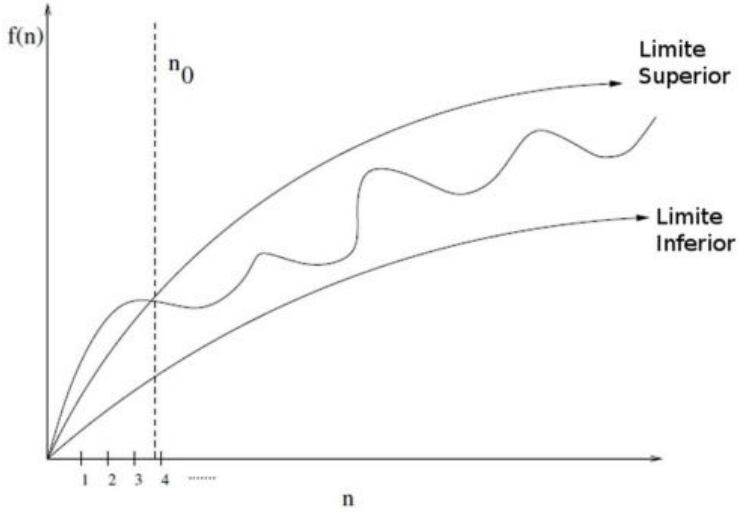
Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	Θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	Θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	Θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	Θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	Θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)


LEGEND
TIME Complexity VS. SPACE Complexity
Good Fair Bad
Good Fair Bad
<BIG-O-CHEATSHEET>
www.bigocheatsheet.com
0(n!) 0(2^n)
0(n^2)
0(n log n)
0(n)
0(1), 0(log n)
Operations
Elements
DATA STRUCTURE Operations
ARRAY SORTING Algorithms
DATA Structure
TIME Complexity
SPACE Complexity
ARRAY Algorithms
Average
Worst
Best
Access Search Insertion Deletion
Array O(1) O(n) O(n) O(n) O(1) O(n) O(n) O(n) O(n)
Stack O(n) O(n) O(1) O(1) O(n) O(n) O(1) O(1) O(n)
Queue O(n) O(n) O(1) O(1) O(n) O(n) O(1) O(1) O(n)
Singly-Linked List O(n) O(n) O(1) O(1) O(n) O(n) O(1) O(1) O(n)
Doubly-Linked List O(n) O(n) O(1) O(1) O(n) O(n) O(1) O(1) O(n)
Skip List O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(n) O(n) O(n) O(n) O(n log(n))
Hash Table N/A O(1) O(1) O(1) N/A O(n) O(n) O(n) O(n)
Binary Search Tree O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(n) O(n) O(n) O(n) O(n)
Cartesian Tree N/A O(log(n)) O(log(n)) O(log(n)) N/A O(n) O(n) O(n) O(n)
B-Tree O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(n)
Red-Black Tree O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(n)
Splay Tree N/A O(log(n)) O(log(n)) O(log(n)) N/A O(log(n)) O(log(n)) O(log(n)) O(n)
AVL Tree O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(n)
KD Tree O(log(n)) O(log(n)) O(log(n)) O(log(n)) O(n) O(n) O(n) O(n) O(n)
Quicksort Ω(n log(n)) Θ(n log(n)) O(n^2) O(log(n))
Mergesort Ω(n log(n)) Θ(n log(n)) O(n log(n)) O(n)
Timsort Ω(n) Θ(n log(n)) O(n log(n)) O(1)
Heapsort Ω(n log(n)) Θ(n log(n)) O(n log(n)) O(1)
Bubble Sort Ω(n) Θ(n^2) O(n^2) O(1)
Insertion Sort Ω(n) Θ(n^2) O(n^2) O(1)
Selection Sort Ω(n^2) Θ(n^2) O(n^2) O(1)
Tree Sort Ω(n log(n)) Θ(n log(n)) O(n^2)t O(n)
Shell Sort Ω(n log(n)) Θ(n(log(n))^2) O(n(log(n))^2) O(1)
Bucket Sort Ω(n+k) Θ(n+k) O(n^2) O(n)
Radix Sort 1 10 100 Ω(nk) Θ(nk) O(nk) O(n+k)
Counting Sort Ω(n+k) Θ(n+k) O(n+k) O(k)
Cubesort Ω(n) Θ(n log(n)) O(n log(n)) O(n)

UniSENAI

Limites superior e inferior

- Todas as ordens de complexidade vistas definem o **Limite Superior (Upper Bound)** dos algoritmos:
  - Qualquer que seja o tamanho da entrada, o tempo de execução crescerá com velocidade igual ou inferior a apontada pela análise de complexidade.
  - Algumas otimizações podem ser feitas para melhorar o limite superior.
- Existem, porém, os **Limites Inferiores (Lower Bound)** para certos problemas, que são pontos a partir dos quais não é mais possível otimizar uma solução algorítmica.



UniSENAI

Exercício coletivo - soma de elementos de um vetor

```java
public class SomaVetor {

    public static int somaVetor(int[] vetor) {
        int soma = 0;
        for (int elemento : vetor) {
            soma += elemento;
        }
        return soma;
    }

    Run | Debug
    public static void main(String[] args) {
        int[] vetor = {1, 2, 3, 4, 5};
        int resultado = somaVetor(vetor);
        System.out.println("Soma: " + resultado);
    }
}
```

- Por onde começamos? Veja que para calcular a complexidade deste programa precisamos avaliar o método 'main', que faz uma chamada à outro método!
  - $1 + [1+n*(1)+1] + 1 \Rightarrow O(n)$

UniSENAI

Exercício coletivo - multiplicação de matrizes

```java
public class MultiplicacaoMatrizes {

    public static int[][] multiplicaMatrizes(int[][] A, int[][] B, int n) {
        int[][] C = new int[n][n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                for (int k = 0; k < n; k++) {
                    C[i][j] += A[i][k] * B[k][j];
                }
            }
        }
        return C;
    }

    Run | Debug
    public static void main(String[] args) {
        int[][] A = {{1, 2}, {3, 4}};
        int[][] B = {{5, 6}, {7, 8}};
        int[][] resultado = multiplicaMatrizes(A, B, n:2);

        System.out.println("Matriz resultante:");
        for (int i = 0; i < resultado.length; i++) {
            for (int j = 0; j < resultado[i].length; j++) {
                System.out.print(resultado[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

UniSENAI

Oportunidade - English Online 2026 - British Council

- **Que tal começar o ano batendo a sua meta do ano de estudar Inglês?** 🗺️📚
- Se você está procurando fortalecer seu inglês e melhorar suas perspectivas de empregabilidade, o Banco Santander e o British Council estão oferecendo 10.000 vagas para que você possa fazer isso de casa e no seu próprio ritmo, durante 8 semanas. Além disso, se você for um dos 1.500 alunos com melhor desempenho, receberá 20 semanas adicionais de formação com aulas on-line com um professor. Inscreva-se agora! Ah! Você não precisa ter um diploma universitário nem ser cliente do Banco Santander.
- https://app.santanderopenacademy.com/pt-BR/program/english-online-2026-2nd-call

Terá algum custo?

Sem nenhum custo. Os cursos são financiados pelo Banco Santander. Se desejar, ao final do curso, você poderá se inscrever por conta própria para o próximo nível, mas essa opção é totalmente pessoal e você não terá nenhuma pressão para isso.



UniSENAI

Dúvidas? 😱

- **AVA** (https://ava.sesisenai.org.br/login/index.php)
- **E-mail** (matheus.krueger@edu.sc.senai.br)
- **WhatsApp** (https://wa.me/554733219647)

- **Monitoria**
  - Sábados, das 8h às 12h, na sala HUB.

"A persistência é o caminho do êxito."
Charles Chaplin